RATAN TATA MAHARASHTRA STATE SKILLS UNIVERSITY

School of Science Engineering and Technology Skills

Department of Engineering

Software Requirements Specification

for

Fully Optimized Time Table Creation System

Version 1.0

Prepared by

Aditya Jagadale, Pratik Kawade, Punyam Shah

Date 25/02/2026

INDEX

1	Introduction
1.1	Purpose
1.2	Scope
1.3	Definitions, Acronyms, and Abbreviations
1.4	References
1.5	Overview of Document
2	Overall Description
2.1	Product Perspective
2.2	Product Functions
2.3	User Classes and Characteristics
2.4	Operating Environment
2.5	Design and Implementation Constraints
2.6	Assumptions and Dependencies
3	SYSTEM FEATURES AND REQUIREMENTS
3.1	Authentication Module
3.1.1	Login
3.1.2	Session & Logout
3.2	Department Management (Admin)
3.3	Course Management (Admin)

3.4	**Teacher Management (Admin)**
3.5	**Classroom Management (Admin)**
3.6	**Batch Management (Admin)**
3.7	**Timetable Generation (Admin)**
3.7.1	**Triggering Generation**
3.7.2	**Generation Results**
3.7.3	**Dashboard Semester Cards**
3.8	**Timetable Viewing & Editing (Admin)**
3.9	**Change Request Management (Admin)**
3.10	**Teacher — Personal Timetable**
3.11	**Teacher — Lecture Management**
3.12	**Teacher — Change Requests**
3.13	**Student — Timetable View**
3.14	**Student — Course List**
3.15	**Student — Elective Enrollment**
3.16	**Scheduling Algorithm (Generator)**
3.16.1	**Session Building**
3.16.2	**Hard Constraints (must never be violated)**
3.16.3	**Soft Constraints (scored; higher = better)**
3.16.4	**Algorithm Logic**
3.17	**Axios Api**

4	**Interfaces**
4.1	**User Interfaces**
4.1.1	**General User Interface Characteristics**
4.1.2	**Navigation Structure**
4.1.3	**Common Interface Components**
4.1.4	**Timetable Grid Interface**
4.1.5	**Dashboard Interface**
4.2	**Hardware Interfaces**
4.3	**Software Interfaces**
4.3.1	**4.3.1 Node.js Runtime**
4.3.2	**Express.js Framework**
4.3.3	**React Framework**
4.3.4	**Axios HTTP Client**
4.3.5	**Web Browser Environment**
4.4	**Communication Interfaces**
4.4.1	**HTTP Protocol**
4.4.2	**Axios API Design**
4.4.3	**Error Response Format**
5	**Non-Functional Requirements**
5.1	**Performance Requirements**
5.3	**Reliability Requirements**

5.4	Maintainability Requirements
5.5	Scalability Requirements
5.6	Availability Requirements
6	Data REQUIREMENTS
6.1	User
6.2	Department
6.3	Course
6.4	Teacher
6.5	Classroom
6.6	Batch
6.7	TimetableEntry
6.8	Enrollment
6.9	ChangeRequest
7	Diagram

1.Introduction

1.1 Purpose

This Software Requirements Specification document provides a comprehensive and detailed description of the requirements for the University Timetable Creation and Management System. The purpose of this document is to serve as a definitive reference for all stakeholders involved in the development, deployment, and maintenance of the system. This includes software developers, quality assurance engineers, project managers, system administrators, and end users who will interact with the system.

The document aims to clearly articulate what the system shall do, how it shall behave under various conditions, and what constraints govern its operation. By establishing a common understanding of system requirements, this document minimizes ambiguity, reduces the risk of scope creep, and provides a baseline against which the delivered system can be validated.

This SRS has been prepared following industry-standard practices for requirements documentation and adheres to the principles of clarity, completeness, consistency, and verifiability. Each requirement stated herein is intended to be testable, meaning that a specific test case or procedure can be designed to verify whether the requirement has been satisfied.

1.2 Scope

The University Timetable Creation and Management System is a web-based application designed to automate and streamline the complex process of creating, managing, and distributing academic timetables within a university or higher education institution. The system addresses the multifaceted challenge of scheduling courses, assigning teachers, allocating classrooms, and managing student batches while satisfying numerous constraints and optimization objectives.

The scope of this system encompasses the following major functional areas:

Academic Resource Management: The system provides comprehensive management capabilities for core academic resources including departments, courses, teachers, classrooms, and student batches. Each resource type can be created, viewed, updated, and deleted through intuitive user interfaces. The system maintains relationships between these resources and enforces referential integrity to ensure data consistency.

Automated Timetable Generation: At the heart of the system lies a sophisticated timetable generation engine that employs constraint satisfaction programming techniques with backtracking to produce conflict-free schedules. The algorithm considers multiple hard constraints that must be satisfied and soft constraints that optimize the quality of the generated schedule.

Multi-User Role-Based Access: The system supports three distinct user roles with differentiated access privileges and functional capabilities. Administrators have full system access for configuration and management. Teachers can view their schedules and request modifications. Students can view their class schedules and manage elective course enrollments.

Change Request Workflow: The system implements a formal workflow for managing schedule change requests initiated by teachers. This includes request submission, administrative review, approval or rejection, and corresponding timetable updates.

Elective Course Management: Students can browse available elective courses, enroll in courses of their choice, and withdraw from enrolled courses. The system filters timetable displays to show only relevant courses based on enrollment status.

The system is designed to operate as a single-tenant application suitable for deployment within a single educational institution. The current version does not support multi-tenant deployment, integration with external student

information systems, or mobile application interfaces, although the architecture does not preclude future extensions in these directions.

1.3 Definitions, Acronyms, and Abbreviations

Academic Year: A period of formal instruction typically spanning twelve months, divided into semesters. This system assumes a maximum of four academic years for any program.

Administrator (Admin): A system user with the highest level of access privileges, responsible for managing all academic resources, generating timetables, and approving or rejecting change requests.

API (Application Programming Interface): A set of definitions and protocols for building and integrating application software. In this system, the API refers to the RESTful endpoints exposed by the server for client-server communication.

Batch: A subdivision of students within a department and year level, typically used for laboratory sessions where smaller group sizes are required. Batches are identified by section letters such as A, B, C.

Change Request: A formal request submitted by a teacher to modify an existing timetable entry, which may involve rescheduling to a different day and time, canceling a session, or requesting a room change.

Constraint Satisfaction Problem (CSP): A mathematical problem involving finding values for a set of variables that satisfy a collection of constraints. In this system, timetable generation is modeled as a CSP.

Course: An academic subject offered within a department, characterized by its code, credit hours, lecture and laboratory requirements, and assigned instructor.

CRUD: An acronym representing the four basic operations of persistent storage: Create, Read, Update, and Delete.

Department: An academic unit within the university responsible for offering courses and programs in a specific discipline.

Elective Course: A course that students may choose to enroll in based on their interests, as opposed to mandatory courses that all students in a program must complete.

Enrollment: The act of a student registering for an elective course, which affects which timetable entries are visible to that student.

Hard Constraint: A constraint that must be satisfied for a timetable to be valid. Violations of hard constraints result in conflicts that make the schedule unusable.

JSON (JavaScript Object Notation): A lightweight data interchange format used for storing and transmitting data. This system uses JSON files for data persistence.

Laboratory (Lab): A practical session for a course, typically requiring specialized facilities and smaller student groups. Lab sessions often span multiple consecutive time slots.

Lecture: A theoretical teaching session for a course, typically delivered in a classroom to all students in a batch or across batches.

MRV (Minimum Remaining Values): A heuristic used in constraint satisfaction problems that selects the variable with the fewest legal values remaining. In this system, it prioritizes scheduling the most constrained sessions first.

REST (Representational State Transfer): An architectural style for designing networked applications, using standard HTTP methods for communication between client and server.

Semester: A half-year academic term. This system supports eight semesters across four years, with odd semesters in the first half of each year and even semesters in the second half.

Session: In the context of timetable generation, a session refers to a single schedulable unit derived from a course's weekly lecture or laboratory requirements.

Slot: A fixed time period during the day when classes can be scheduled. This system defines seven one-hour slots from 9:00 AM to 5:00 PM with a lunch break from 1:00 PM to 2:00 PM.

Soft Constraint: A preference that should be satisfied if possible but whose violation does not make the schedule invalid. Soft constraints are used to optimize schedule quality.

SPA (Single Page Application): A web application that interacts with the user by dynamically rewriting the current page rather than loading entire new pages from the server.

Student: A system user who can view their class schedule and manage elective course enrollments.

Teacher: A system user who can view their teaching schedule and submit change requests. Teachers are linked to courses and departments.

Timetable Entry: A single scheduled session in the timetable, specifying the course, teacher, classroom, day, time slot, and associated batch.

User: Any person who interacts with the system, classified into one of three roles: Administrator, Teacher, or Student.

1.4 References

This document references the following standards, guidelines, and resources:

IEEE Standard 830-1998: IEEE Recommended Practice for Software Requirements Specifications. This standard provides guidance on the content and qualities of a good software requirements specification.

IEEE Standard 29148-2018: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. This is the updated standard that supersedes IEEE 830 and provides comprehensive guidance on requirements engineering.

The Document Object Model (DOM) Level 3 Core Specification. World Wide Web Consortium. This specification is relevant to the client-side implementation using React.

ECMAScript 2021 Language Specification. ECMA International. This specification defines the JavaScript language used in both client and server implementations.

Hypertext Transfer Protocol (HTTP/1.1) Specification. RFC 7230-7235. Internet Engineering Task Force. This defines the communication protocol used between client and server.

JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259. Internet Engineering Task Force. This defines the data format used for storage and API communication.

React Documentation. Meta Platforms, Inc. This documentation covers the React library used for building the user interface.

Express.js Documentation. OpenJS Foundation. This documentation covers the Express framework used for building the server application.

Node.js Documentation. OpenJS Foundation. This documentation covers the Node.js runtime used for server-side JavaScript execution.

1.5 Overview of Document

This Software Requirements Specification is organized into seven major sections, each addressing a distinct aspect of the system requirements.

Section 1 :which you are currently reading, introduces the document by explaining its purpose, defining the scope of the system, establishing terminology, listing references, and providing this structural overview.

Section 2 : provides an overall description of the system, placing it in context relative to other systems and stakeholders. This section describes the product's position in the larger ecosystem, summarizes its major functions, characterizes the users who will interact with it, specifies the operating environment, identifies constraints on design and implementation, and states assumptions and dependencies.

Section 3: forms the core of this document, presenting detailed functional requirements organized by system modules. Each module is described in terms of its purpose, the specific requirements it must satisfy, the inputs it accepts, the outputs it produces, and the business rules that govern its behavior. This section covers authentication, resource management, timetable generation, change requests, and enrollment management.

Section 4: specifies external interface requirements, describing how the system interacts with users through its user interface, with hardware devices, with other software systems, and over communication networks.

Section 5: presents non-functional requirements that specify quality attributes the system must exhibit. These include performance characteristics, security measures, reliability expectations, availability targets, maintainability considerations, and portability requirements.

Section 6: details data requirements, providing a comprehensive data dictionary that defines all data entities, their attributes, and relationships. This section also specifies data integrity constraints and data retention policies.

Section 7: contains appendices with supplementary technical information, including a detailed specification of the timetable generation algorithm, a reference list of business rules, and a catalog of error codes and messages.

2 Overall Description

2.2 Product Perspective

The University Timetable Creation and Management System is designed as a self-contained web application that operates independently without requiring integration with external systems. It is intended to serve as the primary tool for academic scheduling within an educational institution, replacing manual spreadsheet-based processes or legacy scheduling software.

The system follows a client-server architecture with clear separation between the presentation layer, application logic, and data storage. The client component is a single-page application that runs in web browsers and communicates with the server through RESTful API endpoints. The server component handles business logic, data validation, and persistence operations.

From a deployment perspective, the system operates as a single instance serving all users within an institution. It does not currently support multi-tenant deployment where multiple institutions share the same system instance with isolated data. However, the modular architecture would facilitate such an extension if required in the future.

The system does not integrate with external student information systems, human resource systems, or learning management systems. All data regarding departments, courses, teachers, students, and classrooms must be entered directly into the system. This independence simplifies deployment but requires manual data synchronization if the institution maintains parallel systems.

The user interface is designed for desktop and laptop computers with standard web browsers. While the responsive design provides basic functionality on tablet devices, the system is not optimized for mobile

phone screens, and certain features may be difficult to use on smaller displays.

The system assumes a stable network connection between clients and the server. It does not implement offline functionality or local data caching beyond standard browser behavior. Users must have network access to perform any system operations.

2.2 Product Functions

The University Timetable Creation and Management System provides the following major functions, organized by functional area:

User Authentication and Session Management: The system authenticates users based on username and password credentials, maintains user sessions across interactions, enforces role-based access control, and provides secure logout functionality. Each user is assigned one of three roles that determine their access privileges and available features.

Department Administration: The system enables creation and management of academic departments, including their names, short codes, number of years, heads of department, and descriptions. Department data serves as the foundational organizational structure for all other academic entities.

Course Administration: The system supports comprehensive course management including creation of courses with attributes such as name, code, credit hours, weekly lecture and laboratory requirements, duration specifications, and assignment to one or more departments. Courses can be designated as elective to enable student enrollment features.

Teacher Administration: The system manages teacher profiles including personal information, contact details, qualifications, departmental affiliations, and course assignments. When a teacher is added to the system, a corresponding user account is automatically created to enable system access. Teacher assignment to courses is bidirectionally synchronized.

Classroom Administration: The system maintains a registry of physical spaces available for scheduling, categorizing them as lecture halls or laboratories, and recording their capacities, building locations, floor levels, and available facilities.

Batch Administration: The system enables division of student populations into manageable batches for laboratory sessions. Batches are associated with specific departments and year levels. An automated batch splitting feature distributes students evenly across batches based on total enrollment and maximum batch size parameters.

Automated Timetable Generation: The system generates conflict-free timetables using a constraint satisfaction algorithm. The generation process considers teacher availability, room availability, student group constraints, room type requirements, and various optimization criteria. Generation can be performed for individual semesters or entire departments.

Timetable Viewing and Display: The system provides multiple views of timetable data including weekly grid displays showing all sessions across days and time slots, daily views showing sessions for a specific day, and filtered views based on department, semester, batch, or teacher.

Timetable Editing and Maintenance: The system allows manual editing of individual timetable entries to change assigned courses, teachers, or classrooms. Entries can be deleted individually or in bulk for a department and semester combination.

Session Cancellation and Restoration: Teachers can cancel individual class sessions without deleting them from the system. Cancelled sessions are visually distinguished and can be restored to active status when appropriate.

Change Request Submission: Teachers can submit formal requests to reschedule classes to different days or time slots, request cancellation of

classes, or request assignment to different classrooms. Each request includes the affected entry, requested change type, and reason.

Change Request Review and Processing: Administrators can view pending change requests, review the details and reasons provided, and either approve or reject each request. The workflow maintains a complete history of request status changes.

Elective Course Enrollment: Students can view available elective courses for their program, enroll in courses of their choice, and withdraw from previously enrolled courses. The system prevents duplicate enrollments and validates course availability.

Enrollment-Based Schedule Filtering: When students view their timetables, the display is automatically filtered to show only mandatory courses for their department and semester plus elective courses in which they have enrolled. This provides a personalized schedule view.

Statistical Reporting: The system provides summary statistics including counts of departments, courses, teachers, classrooms, and batches, as well as timetable metrics such as scheduled sessions per semester and teacher workload distributions.

2.3 User Classes and Characteristics

The system is designed to serve three distinct classes of users, each with different characteristics, needs, and interaction patterns.

Administrator Users: Administrators are typically staff members responsible for academic scheduling and administration. They may hold positions such as Academic Coordinator, Registrar, or Department Administrator. Administrators require full access to all system functions and bear responsibility for maintaining accurate data and generating valid timetables.

Administrators are expected to have familiarity with academic scheduling concepts, institutional policies regarding class sizes and teaching loads, and

general computer literacy. They do not require technical expertise in programming or database management, but should understand the implications of scheduling constraints and the factors that affect timetable quality.

The typical administrator workflow involves initial data entry for departments, courses, teachers, and classrooms at the beginning of an academic period, followed by timetable generation, review, and adjustment. During the academic term, administrators process change requests and handle exceptions as they arise.

Administrators interact with the system regularly, possibly daily during peak periods such as the start of semesters, and less frequently during stable operation phases. They require efficient interfaces for bulk operations and clear feedback on system status and any errors encountered.

Teacher Users: Teachers are academic staff members who deliver instruction in one or more courses. They access the system primarily to view their teaching schedules and to request modifications when necessary due to conflicts, personal obligations, or other circumstances.

Teachers are expected to have basic computer literacy sufficient for web browsing and form completion. They do not require understanding of the overall scheduling process or administrative functions. The system must present their information in a clear, uncluttered manner that allows quick reference.

The typical teacher workflow involves checking their schedule at the beginning of a term, referring to it periodically throughout the term, and occasionally submitting change requests. Some teachers may also cancel or restore individual class sessions in response to events such as conferences or illness.

Teachers interact with the system occasionally, perhaps weekly or as needed when schedule questions arise. They require a simple, intuitive

interface that provides immediate access to their personal schedule without navigating through unrelated information.

Student Users: Students are individuals enrolled in academic programs who need to know when and where their classes take place. They also manage their participation in elective courses through the enrollment feature.

Students are assumed to be technologically proficient with web applications and comfortable with self-service features. However, they have no need or authority to access administrative functions or modify system data beyond their own elective enrollments.

The typical student workflow involves checking their schedule at the beginning of a term, enrolling in desired elective courses during the enrollment period, and referring to the schedule throughout the term. Students may access the system more frequently during periods of schedule changes or when planning their daily activities.

Students represent the largest user population but have the simplest functional requirements. Their interface emphasizes clarity and quick access to personalized schedule information rather than comprehensive data management capabilities.

2.4 Operating Environment

The University Timetable Creation and Management System operates within the following technical environment:

Server Environment: The server component executes within a Node.js runtime environment, version 14.0 or higher. Node.js provides the JavaScript execution context for running the Express.js web application framework that handles HTTP requests, routes, and middleware processing.

The server requires access to a file system with read and write permissions for the data storage directory. All persistent data is stored as JSON files within this directory. The server process requires sufficient memory to load

and manipulate these JSON files, which grow in proportion to the number of records stored.

The server should be deployed on hardware or virtual infrastructure with at least 2 GB of RAM and 10 GB of storage space for a typical installation. Larger installations with more historical data may require proportionally more resources.

Client Environment: The client application runs within modern web browsers that support ECMAScript 2015 and later JavaScript features, HTML5, and CSS3. Supported browsers include Google Chrome version 80 or higher, Mozilla Firefox version 75 or higher, Microsoft Edge version 80 or higher, and Apple Safari version 13 or higher.

The client requires JavaScript to be enabled in the browser. The application will not function with JavaScript disabled. Certain features rely on session storage for maintaining authentication state across page loads.

Network Environment: The system assumes HTTP or HTTPS communication between clients and server. For production deployment, HTTPS with valid TLS certificates is strongly recommended to protect authentication credentials and user data in transit.

The server exposes a single port for both serving the client application files and handling API requests. The default port is 5000, but this is configurable through environment variables.

Browser Requirements: Client devices should have displays with a minimum resolution of 1024 by 768 pixels for optimal viewing of timetable grids. Lower resolutions can access the system but may require horizontal scrolling to view complete timetable displays.

2.5 Design and Implementation Constraints

Several constraints influence the design and implementation of the system:

Technology Stack Constraints: The decision to use React for the client application and Express.js for the server application constrains the system to JavaScript and its ecosystem. All team members must be proficient in JavaScript and familiar with React patterns including functional components, hooks, and context-based state management.

Data Storage Constraints: The use of JSON files for data storage rather than a relational database imposes certain limitations. Concurrent write operations must be handled carefully to avoid data corruption. Large datasets may experience performance degradation during read and write operations. Complex queries requiring joins across multiple data collections must be implemented in application code rather than leveraging database query optimizers.

The JSON storage approach was chosen for simplicity of deployment and elimination of database administration requirements. For installations with concurrent users or data volumes exceeding approximately 10,000 timetable entries, migration to a database management system should be considered.

Algorithm Constraints: The timetable generation algorithm uses heuristic methods with backtracking to find feasible schedules. While effective for typical academic scenarios, the algorithm does not guarantee optimal solutions in the mathematical sense. Extremely constrained scenarios with many courses, limited rooms, and tight teacher availability may fail to find complete solutions.

The algorithm implements a maximum backtracking limit to prevent excessive computation time. This means very difficult scheduling problems may report partial solutions with some sessions unscheduled. Administrators must review generation results and may need to adjust constraints or manually schedule remaining sessions.

User Interface Constraints: The user interface is designed for desktop browsers and does not provide a dedicated mobile application. While

responsive design techniques provide basic mobile compatibility, certain features such as the timetable grid are difficult to use on small screens. Organizations requiring robust mobile access should consider supplementary solutions.

Scalability Constraints: The system is designed for single-institution deployment serving up to approximately 500 concurrent users. Scaling beyond this level would require architectural changes including database migration, session management updates, and potential introduction of server load balancing.

Security Constraints: The current implementation stores passwords in plain text within the users data file. This approach is acceptable only for demonstration and development purposes. Production deployment must implement proper password hashing using algorithms such as bcrypt before the system is exposed to real users.

2.6 Assumptions and Dependencies

The following assumptions underlie the requirements specified in this document:

Institutional Structure Assumptions: It is assumed that the institution organizes its academic offerings into departments, that each department offers multiple courses, that courses are associated with specific years and semesters, and that students are enrolled in specific departments with defined progression through years and semesters.

Temporal Assumptions: It is assumed that the academic week spans Monday through Saturday with Sunday as a non-teaching day. It is assumed that teaching hours fall between 9:00 AM and 5:00 PM with a lunch break from 1:00 PM to 2:00 PM. Changes to these assumptions would require modifications to system constants and algorithms.

Scheduling Assumptions: It is assumed that each course requires a fixed number of lecture and laboratory sessions per week that remain constant throughout a semester. It is assumed that one teacher is assigned to each

course and handles all sessions for that course. It is assumed that batches within the same department and year level share the same lecture sessions but have separate laboratory sessions.

User Assumptions: It is assumed that all users have access to desktop or laptop computers with modern web browsers and stable internet connectivity. It is assumed that administrators receive appropriate training on system operation and scheduling principles. It is assumed that teacher and student users can navigate basic web interfaces without specialized training.

Technical Dependencies: The system depends on Node.js runtime availability on the server host. The system depends on npm package manager for installing and managing dependencies. The system depends on browser implementations conforming to web standards for JavaScript, HTML, and CSS.

Operational Dependencies: The system depends on administrators to enter accurate data for departments, courses, teachers, and classrooms before timetable generation. The accuracy and quality of generated timetables directly depends on the completeness and validity of input data.

3. System Features & Functional Requirements

3.1 Authentication Module

Description: Provides login, session management, and role-based navigation. The system has no self-registration; accounts are seeded by the administrator.

3.1.1 Login

ID	Requirement	Priority
FR-1.1	The system shall provide a /login page accessible to unauthenticated users	P1
FR-1.2	The login form shall accept **username** and **password** fields; both are required	P1
FR-1.3	On submission the client shall POST /api/login with {username, password}	P1
FR-1.4	The server shall locate a user where both username and password match exactly (case-sensitive)	P1
FR-1.5	On success the server shall return the user object with the password field **stripped**	P1
FR-1.6	The client shall store the user object in sessionStorage under key tt_user	P1
FR-1.7	On success the client shall navigate to /{role} (e.g. /admin, /teacher, /student)	P1
FR-1.8	On failure the server shall return HTTP 401; the client shall display an error banner	P1
FR-1.9	The submit button shall be **disabled** and show "Signing in…" while the request is in-flight	P2

ID	Requirement	Priority
FR-1.10	Three **demo account cards** (Admin, Teacher, Student) shall auto-fill credentials on click	P3

3.1.2 Session & Logout

ID	Requirement	Priority
FR-1.11	On page refresh the system shall restore the session from sessionStorage without re-login	P1
FR-1.12	An authenticated user visiting /login shall be redirected to their role dashboard	P1
FR-1.13	Clicking Logout shall remove tt_user from sessionStorage and redirect to /login	P1
FR-1.14	Admin pages, Teacher pages, and Student pages shall only be accessible to their respective roles	P1

3.2 Department Management (Admin)

Description: Allows the administrator to create and maintain academic departments. Departments are the root entity from which courses, teachers, and batches derive.

ID	Requirement	Priority
FR-2.1	Admin shall view all departments as **visual cards** showing name, code, icon, HOD, course count, teacher count	P1
FR-2.2	Admin shall **search** departments by name or code (case-insensitive, real-time)	P2

ID	Requirement	Priority
FR-2.3	Admin shall **add** a department with: Name (required), Code (required, ≤6 chars, unique), Duration in Years (1–4), HOD (optional), Description (optional)	P1
FR-2.4	The system shall **validate** all required fields and display field-level error messages before saving	P1
FR-2.5	The system shall **reject** a duplicate department code and display the conflicting department name	P1
FR-2.6	The modal form shall show a **live preview** card updating as the user types	P3
FR-2.7	Admin shall **edit** any field of an existing department via the same modal form	P1
FR-2.8	Admin shall **delete** a department; a confirmation dialog shall warn if the department has linked courses or teachers	P1
FR-2.9	Each department card shall display an **auto-generated icon** and **colour theme** derived from its name/code	P3

3.3 Course Management (Admin)

Description: Courses are the atomic schedulable units. Each course defines how many lectures and labs per week, their duration, and which department/teacher it belongs to.

ID	Requirement	Priority
FR-3.1	Admin shall view all courses in a filterable list	P1
FR-3.2	Admin shall filter courses by department, year, and semester	P1
FR-3.3	The course list shall display the resolved teacher name (or "Unassigned" if none)	P2

ID	Requirement	Priority
FR-3.4	Admin shall add a course with: Name, Code, Year (1–N), Semester (1–2N), Department(s) (multi-select), Weekly Lectures (≥0), Weekly Labs (≥0), Lecture Duration (slots), Lab Duration (slots), Assigned Teacher, Is Elective flag, Credits	P1
FR-3.5	All numeric course fields shall be coerced to Number type before persistence	P1
FR-3.6	A course shall support being assigned to multiple departments (departmentIds[])	P2
FR-3.7	Admin shall edit any course field	P1
FR-3.8	Admin shall delete a course	P1
FR-3.9	The system shall support the legacy single departmentId field alongside the new departmentIds[] array for backward compatibility	P2

3.4 Teacher Management (Admin)

Description: Teachers are linked to both courses (as instructors) and user accounts (for login). Creating a teacher auto-provisions their login.

ID	Requirement	Priority
FR-4.1	Admin shall view all teachers; filterable by department	P1
FR-4.2	Admin shall **add** a teacher with: Name, Email, Department(s), Course Assignments	P1
FR-4.3	On teacher creation the system shall **auto-create** a user account: username = email.split('@')[0], password = 'teacher123', role = 'teacher', linkedId = teacher.id	P1

ID	Requirement	Priority
FR-4.4	On creation each assigned course shall have its teacherId updated to this teacher	P1
FR-4.5	Admin shall **edit** a teacher; the system shall diff old vs new courseIds and update course teacherId fields accordingly: removed courses → teacherId = '', added courses → teacherId = teacher.id	P1
FR-4.6	Admin shall **delete** a teacher; the system shall: (1) un-assign the teacher from all courses, (2) delete the teacher record, (3) delete the linked user account	P1

3.5 Classroom Management (Admin)

Description: Classrooms are physical rooms of type lecture or lab with a defined capacity. The scheduling engine selects rooms based on type-matching and capacity.

ID	Requirement	Priority
FR-5.1	Admin shall view all classrooms; filterable by type (lecture / lab)	P1
FR-5.2	Admin shall **add** a classroom with: Name, Type (lecture or lab), Capacity (Number)	P1
FR-5.3	Admin shall **edit** classroom details	P1
FR-5.4	Admin shall **delete** a classroom	P1

3.6 Batch Management (Admin)

Description: Batches subdivide a year's student cohort into groups (A, B, C…) so that the scheduler generates separate, conflict-free timetables per group.

ID	Requirement	Priority
FR-6.1	Admin shall view all batches; filterable by department and year	P1
FR-6.2	Admin shall **manually add** a batch with: Name, Section label, Department, Year, Student Count	P1
FR-6.3	Admin shall trigger **auto-split**: given Total Students and Max Per Batch, the system calculates numBatches = ceil(total/max) and names them A, B, C … with students distributed evenly (remainder allocated to the first batches)	P1
FR-6.4	Auto-split shall **delete** all existing batches for the same department+year before creating new ones	P1
FR-6.5	Admin shall **edit** or **delete** individual batches	P1

3.7 Timetable Generation (Admin)

Description: The core feature. The admin triggers automated timetable generation using a CSP solver. Results are stored and immediately viewable.

3.7.1 Triggering Generation

ID	Requirement	Priority
FR-7.1	Admin shall generate a timetable for a specific **department + semester**	P1
FR-7.2	Admin shall optionally restrict generation to a **specific batch** within a semester	P1
FR-7.3	Admin shall generate timetables for **all semesters** (1–8) of a department in one operation	P1
FR-7.4	Generation mode 'week' schedules across all 6 days; mode 'day' schedules Monday only	P2

ID	Requirement	Priority
FR-7.5	A **confirmation dialog** shall appear before any generation (listing scope)	P2
FR-7.6	During generation the button shall show "Working…" and be disabled	P2

3.7.2 Generation Results

ID	Requirement	Priority
FR-7.7	The API response shall include: placed (number of session-slots placed), errors[] (unplaced sessions), backtrackCount, algorithm name, batches count	P1
FR-7.8	A success toast shall display the number of placed slots	P1
FR-7.9	Each unplaced session shall produce a warning toast listing the course and batch	P1
FR-7.10	New entries shall replace old entries for the same department+semester (or batch) scope; all other entries are preserved	P1

3.7.3 Dashboard Semester Cards

ID	Requirement	Priority
FR-7.11	Each semester card shall display: total courses, scheduled/total coverage, batch list with section labels and student counts, slot count badge	P2
FR-7.12	A "Delete" button on each card shall remove all timetable entries for that semester after confirmation	P1

3.8 Timetable Viewing & Editing (Admin)

ID	Requirement	Priority
FR-8.1	Admin shall select department + semester to view the corresponding weekly grid	P1
FR-8.2	When the timetable contains batch entries a batch selector shall appear; the grid shows only the selected batch (plus non-batch entries)	P1
FR-8.3	Clicking a grid cell shall open an edit modal for that entry	P1
FR-8.4	The edit modal shall allow changing: Course (dept-filtered list), Teacher, Classroom, Session Type; Day and Slot are read-only	P1
FR-8.5	Admin shall save edits (PUT) or delete the entry from the edit modal	P1
FR-8.6	Admin shall view all timetable entries across all departments via the "View All" page	P2

3.9 Change Request Management (Admin)

ID	Requirement	Priority
FR-9.1	Admin shall view all change requests split into Pending and Resolved tables	P1
FR-9.2	Each pending request row shall show: teacher name, request type badge, course code+name, current schedule slot, requested new slot (if reschedule), reason, submission date	P1
FR-9.3	Admin shall Approve or Reject a pending request; a confirmation dialog shall precede the action	P1
FR-9.4	Resolved requests shall show a status badge (green = approved, red = rejected)	P2

3.10 Teacher — Personal Timetable

ID	Requirement	Priority
FR-10.1	A teacher shall view their weekly timetable grid (all 6 days × 7 slots) showing only their assigned sessions	P1
FR-10.2	A teacher shall view their daily timetable filtered by a selectable day	P1
FR-10.3	The timetable shall show only entries with status = 'active'	P1

3.11 Teacher — Lecture Management

ID	Requirement	Priority
FR-11.1	A teacher shall view their full lecture list split into Active and Cancelled sections	P1
FR-11.2	A teacher shall cancel an active lecture (soft-delete: sets status = 'cancelled') after a confirmation dialog	P1
FR-11.3	A teacher shall restore a cancelled lecture (sets status = 'active')	P1
FR-11.4	Cancelled entries shall appear at 60% opacity with a Restore button	P2

3.12 Teacher — Change Requests

ID	Requirement	Priority
FR-12.1	A teacher shall view only their own change requests (filtered by teacherId)	P1

ID	Requirement	Priority
FR-12.2	A teacher shall submit a new change request by selecting a lecture, request type, and reason	P1
FR-12.3	Request types shall be: Reschedule (requires new day + time slot), Cancel (no new slot), Room Change (no new slot)	P1
FR-12.4	The request payload shall include: teacherId, entryId, courseCode, courseName, day, slotLabel, type, reason, and conditionally newDay + newSlot	P1
FR-12.5	Request status badges: pending = amber, approved = green, rejected = red	P2

3.13 Student — Timetable View

ID	Requirement	Priority
FR-13.1	A student shall view their **weekly timetable grid** for their registered department + semester	P1
FR-13.2	A student shall view their **daily timetable** filtered by a selectable day	P1
FR-13.3	Each timetable cell shall show: course name/code, teacher name, classroom name, session type	P1

3.14 Student — Course List

ID	Requirement	Priority
FR-14.1	A student shall view all courses for their department showing: name, code, session type, weekly count, assigned teacher	P2

3.15 Student — Elective Enrollment

ID	Requirement	Priority
FR-15.1	A student shall view all **elective courses** (isElective = true) available for their department and year	P1
FR-15.2	The page shall show two sections: **Enrolled** electives (green border) and **Available** electives (blue border)	P1
FR-15.3	Each elective card shall show: course code, name, instructor name, lectures/week, lab sessions/week (if any), credits (if any), department code badges	P2
FR-15.4	A student shall click **"Enroll in This Course"** to enroll; the button shows "Enrolling…" and is disabled during the request	P1
FR-15.5	The server shall **reject duplicate enrollments** with HTTP 409	P1
FR-15.6	A student shall click **"Drop This Course"** to unenroll	P1
FR-15.7	Summary stat cards shall show total available electives and enrolled count	P2
FR-15.8	An empty state message shall appear if no electives are available for the student's dept+year	P2

3.16 Scheduling Algorithm (Generator)

Description: The CSP-based timetable engine is the heart of the system. It transforms course configurations into conflict-free timetable entries.

3.16.1 Session Building

ID	Requirement	Priority
FR-16.1	For each course × each batch, the engine shall create N lecture sessions (N = weeklyLectures) each with slotCount = lectureDuration	P1
FR-16.2	For each course × each batch, the engine shall create M lab sessions (M = weeklyLabs) each with slotCount = labDuration	P1
FR-16.3	If no batches exist for a dept+year, courses shall be scheduled as a single unbatched group	P1

3.16.2 Hard Constraints (must never be violated)

ID	Constraint	Rule
FR-16.4	Slot bounds	All slots needed by a session (slotId to slotId+slotCount-1) must be ≤ 7
FR-16.5	Lunch crossing	Multi-slot sessions that span across the lunch break (slot 4 → slot 5) are forbidden
FR-16.6	Teacher conflict	No two sessions in the schedule may share the same teacherId at the same day+slot
FR-16.7	Room conflict	No two sessions may share the same classroomId at the same day+slot
FR-16.8	Student batch conflict	No two sessions for the same dept+semester+batchId may occupy the same day+slot
FR-16.9	Student group conflict	For unbatched courses, no two sessions for the same dept+semester may occupy the same day+slot
FR-16.10	Room type	Lecture sessions must be placed in rooms of type lecture; lab sessions in rooms of type lab

3.16.3 Soft Constraints (scored; higher = better)

ID	Constraint	Score
FR-16.11	Day spread	New day for course: +15; one existing class: +3; 2+ classes same day: −10
FR-16.12	Daily balance	−3 per existing class the group already has on this day
FR-16.13	Consecutive penalty	≥4 consecutive: −20; ≥3 consecutive: −8
FR-16.14	Time preference	Lecture in morning slots (1–4): +4; Lab in afternoon slots (5–7): +4
FR-16.15	Teacher spread	Teacher free on this day: +6; −2 per existing class on this day
FR-16.16	Room capacity fit	+max(0, 5−floor(excess/10)) for rooms that fit; prefer smallest room
FR-16.17	Day preference	Mon–Fri: +1; Saturday: 0

3.16.4 Algorithm Logic

ID	Requirement	Priority
FR-16.18	Sessions shall be sorted by **MRV**: fewest valid placements first	P1
FR-16.19	The algorithm shall use **backtracking** (up to maxBacktrack=50) to resolve dead-ends	P1
FR-16.20	Multi-slot lab sessions shall be assigned a shared **labGroup ID** linking their individual slot entries	P1

ID	Requirement	Priority
FR-16.21	Unplaceable sessions shall be logged in errors[]; the algorithm shall continue with remaining sessions	P1
FR-16.22	On completion the algorithm shall return {schedule, errors, placed, backtrackCount}	P1

3.17 Axios API

Description: All client-server communication is through an Axios HTTP API on port 5000.

Method	Endpoint	Description
POST	/api/login	Authenticate user
GET/POST	/api/departments	List / Create departments
PUT/DELETE	/api/departments/:id	Update / Delete department
GET/POST	/api/courses	List (filtered) / Create courses
PUT/DELETE	/api/courses/:id	Update / Delete course
GET/POST	/api/teachers	List (filtered) / Create teachers
PUT/DELETE	/api/teachers/:id	Update / Delete teacher
GET/POST	/api/classrooms	List (filtered) / Create classrooms

Method	Endpoint	Description
PUT/DELETE	/api/classrooms/:id	Update / Delete classroom
GET/POST	/api/batches	List (filtered) / Create batch
PUT/DELETE	/api/batches/:id	Update / Delete batch
POST	/api/batches/auto-split	Auto-split batch
POST	/api/timetable/generate	Run scheduling algorithm
GET	/api/timetable	Fetch timetable (dept+sem or teacher)
GET	/api/timetable/all	Fetch all entries
PUT/DELETE	/api/timetable/:id	Edit / Delete single entry
POST	/api/timetable/:id/cancel	Soft-cancel entry
POST	/api/timetable/:id/restore	Restore entry
DELETE	/api/timetable/dept/:deptId/semester/:sem	Clear dept+semester
GET/POST	/api/enrollments	List / Create enrollment
DELETE	/api/enrollments/:id	Delete enrollment
POST	/api/enrollments/unenroll	Bulk unenroll

Method	Endpoint	Description
GET/POST	/api/change-requests	List / Create change request
PUT	/api/change-requests/:id	Update change request status
GET	/api/timeslots	Get time slot definitions
GET	/api/days	Get working days list

4.Interfaces

4.1 User Interfaces

4.1.1 General User Interface Characteristics

The user interface follows modern web application conventions with responsive design principles adapted primarily for desktop and laptop computer displays. The interface uses a consistent visual language throughout with standardized colors, typography, spacing, and interactive patterns.

The color scheme uses professional tones with a primary color for actions and emphasis, neutral colors for backgrounds and containers, success colors for confirmations and positive status, warning colors for alerts and cautions, and danger colors for destructive actions and error states.

Typography uses the Inter font family loaded from Google Fonts, providing excellent readability across different screen sizes and resolutions. Font weights range from regular for body text through bold for emphasis and headings.

All interactive elements provide visual feedback on hover and focus states. Buttons display cursor changes and color variations when hovered. Form fields display focus rings when selected. Clickable elements throughout the application follow consistent interaction patterns.

4.1.2 Navigation Structure

The application uses a sidebar navigation pattern where a persistent vertical menu remains visible on the left side of the screen while content areas occupy the remaining space to the right.

The sidebar displays the application logo and name at the top, followed by the current user's name and role badge, then navigation links specific to the user's role, and finally a logout button at the bottom.

Navigation links are organized by functional area with icons accompanying text labels. The currently active section is highlighted within the navigation list to maintain user orientation.

4.1.3 Common Interface Components

Data Tables: Lists of records are displayed in tabular format with sortable columns, pagination controls for large datasets, and action buttons for each row. Tables support responsive behavior by scrolling horizontally on narrow viewports.

Modal Dialogs: Create and edit operations appear in modal overlays that dim the background and focus attention on the form. Modals include a header with title, a scrollable content area for forms, and a footer with cancel and submit buttons.

Form Fields: Input fields include labels, placeholders, validation messages, and appropriate input types for different data. Select fields provide dropdown options with search capability for long lists.

Toast Notifications: Brief feedback messages appear in the corner of the screen for action confirmations and error notifications. They automatically dismiss after a few seconds but can be manually dismissed by clicking.

Confirmation Dialogs: Destructive actions such as delete operations prompt for confirmation before proceeding with a clear question and cancel versus confirm button options.

4.1.4 Timetable Grid Interface

The timetable grid is a specialized component displaying the weekly schedule in a matrix format. Columns represent days from Monday through Saturday. Rows represent time slots from the first morning slot through the final afternoon slot.

Each cell can contain zero, one, or multiple session cards depending on scheduling density. Session cards display essential information including course code, teacher name abbreviated, and room name. Color coding distinguishes session types.

A fixed row indicates the lunch break period, displayed differently to show it is not schedulable. Row headers show time ranges for each slot. Column headers show day names.

The grid supports click interactions where clicking an empty cell may offer session creation and clicking a session card opens editing options.

4.1.5 Dashboard Interface

The administrative dashboard provides an operational overview with statistics cards showing counts of key entities, a department selector for timetable generation, and action buttons for generation operations.

Statistics cards display numbers with labels and icons representing departments, courses, teachers, classrooms, and batches at a glance.

The generation area allows department selection through a searchable dropdown, displays semesters available for the selected department, shows current schedule status per semester, and provides buttons to generate individual semesters or all semesters together.

4.2 Hardware Interfaces

The University Timetable Creation and Management System is a software application that does not directly interface with specialized hardware devices. All hardware interaction occurs through standard interfaces provided by the operating system and web browsers.

The server component interacts with storage hardware through the file system abstraction provided by the Node.js runtime and underlying operating system. This includes reading and writing JSON data files to persistent storage.

The client component interacts with display hardware through the browser's rendering engine. Screen resolution affects the usability of certain features, with a minimum recommended resolution of 1024 by 768 pixels for optimal timetable grid viewing.

Client input devices including keyboard and mouse or trackpad are required for form completion and navigation. The interface assumes standard pointing

device interactions and does not specifically accommodate voice input or alternative input methods.

Network interface hardware is required for communication between clients and server using TCP/IP protocols over Ethernet or wireless connections.

4.3 Software Interfaces

4.3.1 Node.js Runtime

The server component requires Node.js version 14.0 or higher as its execution environment. Node.js provides the JavaScript runtime, event loop, and core modules including the file system module for data persistence and the HTTP module underlying the Express framework.

Node.js is available for major operating systems including Windows, macOS, and Linux distributions. The application does not depend on operating system specific features and should run equivalently on any supported platform.

4.3.2 Express.js Framework

The server application uses Express.js version 4.x as its web application framework. Express provides HTTP request handling, routing, middleware support, and response generation capabilities.

Key Express middleware used includes the JSON body parser for request payload processing and CORS middleware for cross-origin request handling during development.

4.3.3 React Framework

The client application uses React version 18.x as its user interface framework. React provides component-based architecture, virtual DOM rendering, hooks for state management, and a declarative approach to building user interfaces.

React Router DOM version 6.x provides client-side routing, enabling navigation between different views without full page reloads.

4.3.4 Axios HTTP Client

The client application uses Axios as its HTTP client library for communicating with the server API. Axios provides promise-based request handling, request and response interceptors, and automatic JSON transformation.

4.3.5 Web Browser Environment

The client application runs within web browser environments. Supported browsers include Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari in their recent versions as specified in the operating environment section.

The application depends on browser-provided APIs including session storage for authentication state, the XMLHttpRequest API or Fetch API underlying Axios communication, and standard DOM manipulation APIs.

4.4 Communication Interfaces

4.4.1 HTTP Protocol

All communication between client and server uses the HTTP protocol version 1.1 or higher. During production deployment, HTTPS with TLS encryption should be used to protect data in transit.

The server listens on a configurable port, defaulting to port 5000. The client constructs requests to this port based on the configured base URL.

4.4.2 Axios API Design

The API follows REST architectural principles with resources identified by URLs, standard HTTP methods indicating operations, and JSON payload formats for request and response bodies.

Resource URLs follow a consistent pattern with /api as the common prefix, followed by the resource name in plural form. For example, /api/departments for the department collection and /api/departments/:id for individual department resources.

HTTP GET requests retrieve resources without side effects. HTTP POST requests create new resources. HTTP PUT requests update existing resources. HTTP DELETE requests remove resources.

Request bodies for POST and PUT operations use JSON format with Content-Type header set to application/json. Response bodies use JSON format for both successful responses and error responses.

4.4.3 Error Response Format

API error responses use appropriate HTTP status codes including 400 for bad request, 401 for unauthorized, 404 for not found, and 500 for server errors.

Error response bodies include a JSON object with an error property containing a descriptive message suitable for display to users.

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement	Target
NFR-P1	Timetable generation for a single dept+semester (≤30 courses, 5 batches)	< 10 seconds
NFR-P2	All CRUD endpoints (create, read, update, delete)	< 500 ms response time
NFR-P3	React SPA initial load (First Contentful Paint)	< 3 seconds on broadband
NFR-P4	Client-side memo hooks (useMemo) shall not recompute unless dependencies change	Must not trigger extra renders
NFR-P5	Department dashboard paginates at 6 per page to avoid rendering too many cards	Max 6 cards per render

5.2 Security Requirements

ID	Requirement	Priority
NFR-S1	The password field shall never be returned in any API response	Critical
NFR-S2	Session data (tt_user) shall be stored in sessionStorage (cleared on tab close)	P1
NFR-S3	CORS policy shall be restricted to known origins in production	P1
NFR-S4	Default teacher password (teacher123) shall be documented as a known credential	P1
NFR-S5	Production requirement: passwords must be hashed using bcrypt; authentication must use JWT or signed session cookies	P1 (future)

5.3 Reliability Requirements

ID	Requirement	Priority
NFR-R1	All JSON data files shall be auto-initialised to [] on first server start if missing	P1
NFR-R2	Timetable generation shall be atomic: old entries are only replaced after successful algorithm completion	P1
NFR-R3	Deleting a teacher shall cascade: all linked courses and user accounts are cleaned up in the same request	P1
NFR-R4	The scheduling algorithm shall never throw on unsatisfiable constraints; unplaceable sessions are returned in errors[]	P1
NFR-R5	Duplicate enrollments shall be prevented at the server with HTTP 409	P1

ID	Requirement	Priority
NFR-R6	All entities shall be assigned unique prefixed IDs at creation time	P1

5.4 Maintainability Requirements

ID	Requirement
NFR-M1	Server code shall be separated into three modules: routes (index.js), engine (generator.js), data access (db.js)
NFR-M2	All client HTTP calls shall be centralised in api.js
NFR-M3	Authentication state shall be managed via AuthContext (no prop-drilling)
NFR-M4	Each constraint function in generator.js (checkHardConstraints, scoreSlot, etc.) shall be independently callable for unit testing
NFR-M5	Both old (departmentId) and new (departmentIds[]) data formats shall be supported for backward compatibility

5.5 Scalability Requirements

ID	Requirement
NFR-SC1	The db.js data access interface shall be replaceable with a database-backed implementation without changing routes or generator
NFR-SC2	maxBacktrack in the scheduler shall be a named constant, easily adjustable
NFR-SC3	Client and server shall be independently deployable (static hosting + API server)

5.6 Availability Requirements

ID	Requirement
NFR-AV1	The SPA fallback route ensures no 404 errors on page refresh for any React Router path
NFR-AV2	Server start-up shall succeed even if data files are empty or missing

6. DATA REQUIREMENTS

6.1 User

Field	Type	Required	Description
id	String	Yes	Unique user ID (prefix u-)
username	String	Yes	Login username (unique)
password	String	Yes	Plain-text password (dev only)
role	String	Yes	'admin', 'teacher', or 'student'
name	String	Yes	Display name
linkedId	String	No	Teacher record ID (for teacher accounts)
departmentId	String	No	Department (for student accounts)
year	Number	No	Academic year 1–N (for students)
semester	Number	No	Semester 1–2N (for students)

6.2 Department

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix dept-)
name	String	Yes	Full department name
code	String	Yes	Short code ≤6 chars, unique
years	Number	Yes	Programme duration (1–6), default 4
hod	String	No	Head of Department name
description	String	No	Free-text description

6.3 Course

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix c-)
name	String	Yes	Course full name
code	String	Yes	Course code (e.g. CS201)
departmentIds	String[]	Yes	One or more department IDs
year	Number	Yes	Academic year the course belongs to
semester	Number	Yes	Semester number
teacherId	String	No	Assigned teacher ID
weeklyLectures	Number	Yes	Number of lecture sessions per week

Field	Type	Required	Description
weeklyLabs	Number	Yes	Number of lab sessions per week
lectureDuration	Number	Yes	Slots per lecture (default 1)
labDuration	Number	Yes	Slots per lab (default 2)
isElective	Boolean	No	Whether students can choose to enroll
credits	Number	No	Credit hours

6.4 Teacher

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix t-)
name	String	Yes	Teacher full name
email	String	Yes	Email (used to derive username)
departmentIds	String	No	Department associations
courseIds	String	No	Courses they teach

6.5 Classroom

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix r-)
name	String	Yes	Room name/number
type	String	Yes	'lecture' or 'lab'

Field	Type	Required	Description
capacity	Number	Yes	Maximum occupancy

6.6 Batch

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix b-)
name	String	Yes	Display name (e.g. "Batch A")
section	String	Yes	Section letter (A, B, C…)
departmentId	String	Yes	Owning department
year	Number	Yes	Academic year
studentCount	Number	Yes	Number of students in batch

6.7 TimetableEntry

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix tt-)
courseId	String	Yes	Course reference
courseName	String	Yes	Denormalised course name (+ batch label)
courseCode	String	Yes	Denormalised course code
teacherId	String	No	Assigned teacher
classroomId	String	Yes	Room reference
classroomName	String	Yes	Denormalised room name

Field	Type	Required	Description
day	String	Yes	Day of week (Monday–Saturday)
slotId	Number	Yes	Slot index 1–7
slotLabel	String	Yes	Human-readable time (e.g. "09:00 – 10:00")
startTime	String	Yes	ISO time string (e.g. "09:00")
endTime	String	Yes	ISO time string (e.g. "10:00")
type	String	Yes	'lecture' or 'lab'
departmentId	String	Yes	Parent department
semester	Number	Yes	Semester number
year	Number	Yes	Academic year
status	String	Yes	'active' or 'cancelled'
batchId	String	No	Batch reference (null if no batches)
batchSection	String	No	Section label
labGroup	String	No	Shared ID linking multi-slot lab entries

6.8 Enrollment

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix enr-)
userId	String	Yes	Student user ID
courseId	String	Yes	Elective course ID
enrolledAt	String	Yes	ISO 8601 timestamp

6.9 ChangeRequest

Field	Type	Required	Description
id	String	Yes	Unique ID (prefix cr-)
teacherId	String	Yes	Requesting teacher
teacherName	String	Yes	Denormalised teacher name
entryId	String	Yes	Affected timetable entry
courseCode	String	No	Denormalised course code
courseName	String	No	Denormalised course name
day	String	Yes	Current day
slotLabel	String	Yes	Current slot
type	String	Yes	'reschedule', 'cancel', or 'room-change'
reason	String	No	Rationale for the request
newDay	String	No	Requested new day (reschedule only)
newSlot	String	No	Requested new slot (reschedule only)
status	String	Yes	'pending', 'approved', 'rejected'
createdAt	String	Yes	ISO 8601 creation timestamp